# Astro326 Lab 3

Samuel Jediah Gultom[1], Tai Chung[2], Adrien Bonansea[3]
[1]Department of Astronomy and Astrophysics, University of Toronto
Contact: samuel.gultom@mail.utoronto.ca
[2]Department of Astronomy and Astrophysics, University of Toronto
Contact: tai.chung@mail.utoronto.ca
[3]Department of Astronomy and Astrophysics, University of Toronto
Contact: adrien.bonansea@mail.utoronto.ca

October 2024

## 1 Abstract

In this study, we analyzed the spectra of 2 different elements to conducted sky observations of six different star types using the LISA spectroscope, with the exception of O-type stars. We started by looking at the spectra of hydrogen and a mystery gas (Neon) and performed minor calibrations to obtain accurate results. We then used the LISA telescope to capture images of 6 stars with different star types. We performed a spectral and spacial calibration to plot the spectra's and find the peak wavelength intestines attributed to the star, and so by extension the star types. We tabulated the results and compared, wavelength, temperature and the star types to see the relationship between each of these factors.

## 2 Introduction

Detection and sensors of photons are needed to record electromagnetic waves for data analysis. In the case of this report, we will use it to analyze the spectroscopy, and by extension the composition various bodies. Similarly to our 1st experiment in this course, We will analyze photons and their energies by absorbing light and plotting the spectrum. We can then use known spectra of various elements that we know and find what the body is made of and the percentage composition.

In this report, we will start by looking at specific elements in gaseous form and analyze their spectra. We will then perform calibrations to obtain better and more accurate spectra by utilizing centroids. We can then analyze spectra of selected stellar bodies and use the calibration skills we have learned to obtain accurate spectra. The bodies, in this case stars, we have chosen to analyze are: Vega (A type), Algenib (B type), Iota Piscium (F type), Sadalmelik (G type), Almach (K type), and finally Scheat (M type). As an addition, we will be able to compare the star competitions per star type to see what major percentage differences are present.

We will start the report by looking at how the data and observations were made.

# 3 Data and Observation

In this section we will look at the how spectroscopy works and the math behind the statistical analysis performed in this paper.

## 3.1 Spectroscopy and equipment

Spectroscopy is a scientific method that studies the interaction of matter and light or other types of radiation to identify the properties of the matter. It does this by taking in the light, examining the spectra emitted, absorbed or transmitted. We can then find out the properties of the matter, such as the chemical composition, as we will do later in this report. Usually, spectroscopy is done by using a spectrograph and the spectrometer. The spectrograph focuses takin in the data and producing an image of the spectrum whereas the spectrometer measures light a various wavelength. These devices work similar to CCDs where they detect the photon and make it move along the circuit to detect the energy a every location. More specifically, the light enters an aperture to reach a lens. The lens separate the light ray into multiple rays then passes through another lens for diffraction to separate the light into its original wavelengths. It does this by using the formula:

$$d(sin\alpha + sin\beta) = m\lambda \tag{1}$$

Where:

- d is the grating space
- $\alpha$ is the incident angle
- $\beta$ is the diffraction angle
- m is the diffraction order
- $\lambda$ is the light wavelength

It then uses these wavelength to gives them to the detector for analysis.

## 3.2 Distributions

### 3.2.1 Chi Squared Distribution $\chi^2$

The chi-squared distribution, described by equation 2 below, continuous probability distribution that describes random variables formed from summing the squares of values.

$$f(x;k) = \frac{1}{2^{k/2}\Gamma\left(\frac{k}{2}\right)} x^{\frac{k}{2}-1} e^{-x/2} \quad \text{for } x \geq 0 \tag{2}$$

Where x is the random variable and k is the degrees of freedom.

### 3.2.2 Gaussian/Normal Distribution

The equation for the normal distribution, otherwise know as the gaussian distribution is a statistical method that is symmetrical around the mean of data. The equation below represents is used for the distribution:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \tag{3}$$

The distribution forms a bell curve around the mean. This means there is a high concentration of points around the mean and as you move farther from it (in both directions) the density gets smaller to. We can see this through the standard deviations: approximately 68% of values fall within one standard deviation of the mean, 95% within two, and 99.7% within three, making this a very useful model for natural variations. (Cowan 2023) Additionally, the accuracy of the Gaussian distribution is more accurate the more points there are.

### 3.3 Fitting and Modeling

We will be using the Linear least-squares fitting which describes the statistical method that looks a set of data point and find the best fit. It does this by minimizing the distances between the points and the linear fitted line. The linear fit is shown below:

$$y = mx + b \tag{4}$$

The Least Square fit is below: For a polynomial:

$$y = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots + a_n x^n$$

To find the residuals of this:

$$\chi^2 = \sum_{i=0}^{N} \frac{(y_i - f(x_i))^2}{\sigma_i^2}$$

Where the $y_i$ are the observed value, $f(x_i)$ the polynomial predictions, and $\sigma_i$ the uncertainties. In Python, we will utilize the np.polyfit function to do this for us.

## 4 Data Collection and Reduction

In this lab, we will be using the Red Tide USB650 and LISA spectrograph to make measurement. The Lisa is a mounted telescope at the university of Toronto.

### 4.1 Experiment 1: In Lab Spectroscopy

In the 1st experiment, we record the spectrum of the Helium arc lamp. We then capture the ambient spectrum by following the same procedure but with the lamp off (we will call these dark frames). Finally, we measured the read noise spectrum by putting a cap on the detector which blocks all light. for each recording, we detected 30 images with an exposure time of 150 ms.
Although it may not seem, we still had to do a few data reductions in this first part. When looking at the hydrogen and Neon data, we see that there are extra peaks which are small but present. This is due to the device having a small presence of O2. We will ignore most of these peaks to ensure better analysis.

### 4.2 Experiment 2: Star Observations

For the 2nd experiment, we used the LISA telescope to look at various stars: Vega (A type), Algenib (B type), Iota Piscium (F type), Sadalmelik (G type), Almach (K type), and finally Scheat (M type). Using the software at UofT, we pointed the telescope at a selected star. We tune the image and changed took an exposure time of 15 seconds due to the large amount of noise present. Finally we set a repetition rate of 5 so that we could take use the best data possible. After taking in the

data, we must perform multiple calibrations: a spatial and spectral calibration. For the spacial calibration, we use the polynomial relationship by plotting a model to the centroid positions. We get the centroids by finding the brightest pixel at each Neon row. For the spectral calibration, we fit a polynomial to see the relationship between wavelength and pixel position, which ends up being linear. We can then make a numpy array which is the same shape as the telescope data and edit the star data with the calibrations. Figure 1 below is an image of the Neon graph we will use to identify pixels for calibration:

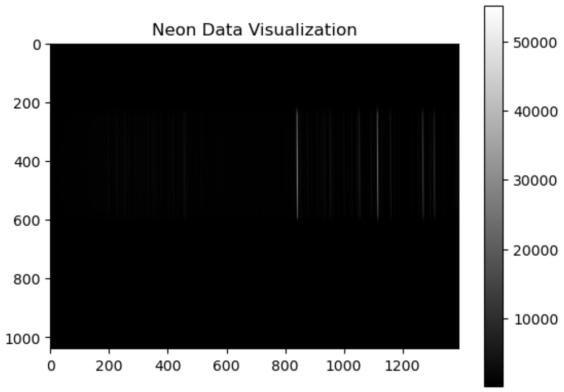


Figure 1: Figure 1: Raw Neon lines

# 5 Data Analysis

## 5.1 In lab Analysis

Figure 1 displays our plot for the H spectrum after the reductions have been made. We can compare this to figure 2, which is the H spectrum plotted from the NIST data. We see that our main peak matches the reference peak but has a a slightly wider base. The 2nd main peak is also a match with a similar wider base. On the other hand, only 1 of the small peaks is present for us and the 2 others do not belong to H but O. This makes sense as we have taken this into account in the extra peaks we were talking about in the modeling.

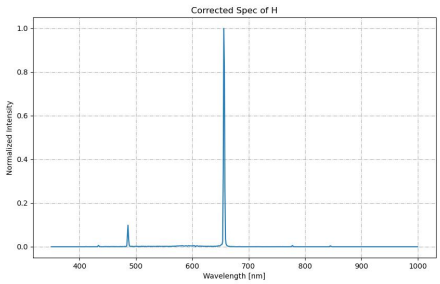


Figure 2: Figure 2: Observed H

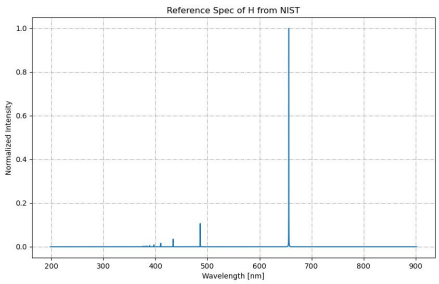


Figure 3: Figure 3: Reference H spectrum from Nist

Our Neon gas data required a few more calibration's due to being shifted at first. Similarly to Hydrogen, the tallest peak has similar hight to the NIST value. However, all the other peaks of Neon were much higher than expected compared to the Nist. We suspect that this is partially due to an instrumental factor or due to the processing of data pushing all the data up.

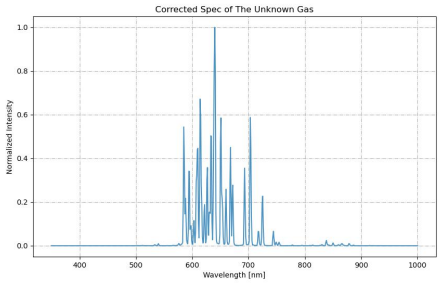


Figure 4: Figure 4: Observed H

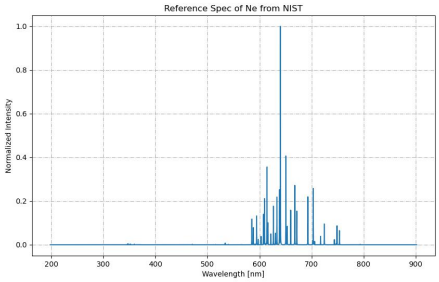


Figure 5: Figure 5: Reference Ne spectrum from Nist

## 5.2 Star observation

As mentioned previously, We performed the In sky observations with 6 different types of stars.

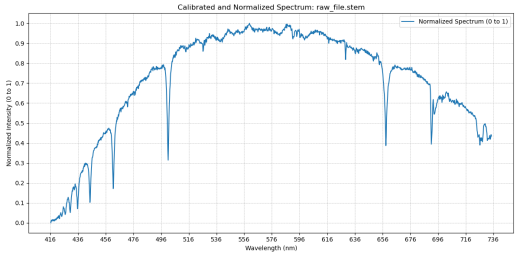


Figure 6: Figure 6: Vega Spectrum (A type)

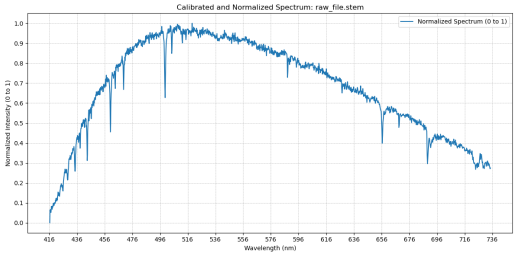


Figure 7: Figure 7: Algenib Spectrum (B type)

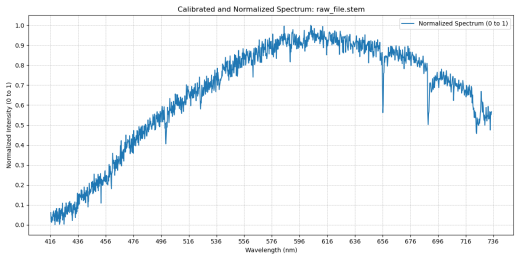


Figure 8: Figure 8: Iota Spectrum (F type)

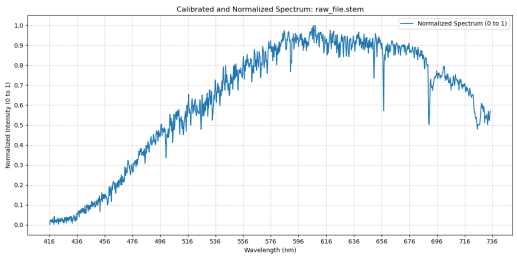


Figure 9: Figure 9: Sada Spectrum (G type)

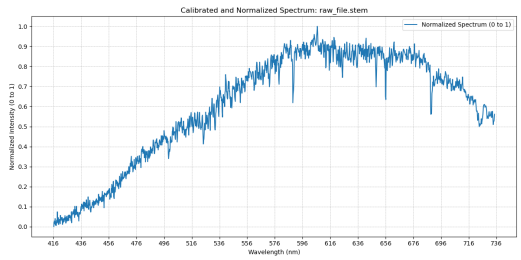


Figure 10: Figure 10: Almach Spectrum (K type)

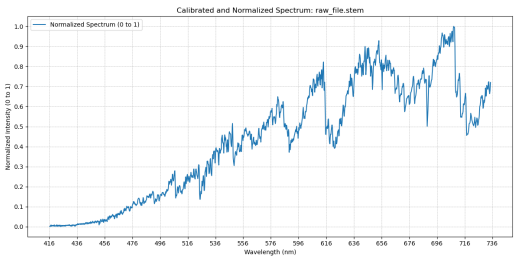


Figure 11: Figure 11: Scheat Spectrum (M type)

As mentioned previously, We performed the In sky observations with 6 different types of stars. The only type that we were not able to capture due to not having any available bright stars that we could capture using LISA was the O type stars. After doing the spectral calibrations and spectral analysis and running the program for each star, we can see their spectra (Figure 5 through 10 inclusive). Sadly, We notice that there is quite a bit of noise in the spectra despite the calibrations. These stars are all from different classes we will be able to look at the spectra and see the clear differences. We are now able confirm their spectral classes and temperature. The names of the star types already give us a good idea on what properties we are expecting. We see that hotter stars, A and B type stars, have a temperature between 30 000K and 7500K, have a larger intensity of at lower wavelengths on the power spectra. As we go down the list of temperature, we can see that the intensity shifts to lower wavelengths. (Cannon et all).
This is shown in table 1:

Table 1: Spectral Types and Temperatures of Stars

| **Spectral Type** | **Temperature Range (K)** | **highest concentration wavelength** |
|---|---|---|
| B | $10,000 - 30,000$ | 496 |
| A | $7,500 - 10,000$ | 550 |
| F | $6,000 - 7,500$ | 590 |
| G | $5,200 - 6,000$ | 600 |
| K | $3,700 - 5,200$ | 606 |
| M | $< 3,700$ | 700 |

We can then conclude that Hotter stars will have shorter peak emission concentration at lower wavelength.
Finally, using our knowledge from experiment 1, we can find look at these star spectrum and find their compositions. Unfortunately, more data and research is required for a detailed look at star composition. On the other hand, we can also infer that there is a high concentration of Hydrogen, especially in lower star types due to having a large peak at around where the large hydrogen peak is. If more research is made, we can then find percentage compositions, and other present elements that are present.

# 6 Discussion and Conclusion

In this report, we looked at what spectroscopy is and how we use it in the context of analyzing spectra. We then looked at the devices that will be used to see how we will collect data and what kind of calibration we will need to do. In the 1st experiment, we found that our hydrogen spectrum was quite similar to the one provided by NIST apart from the extra small peaks which originated from extra oxygen affecting the system. We then did the same for the mystery gas which turned out to be Neon. The peaks measured for our neon were quite a bit higher than the ones given by NIST. We theorized that the source of this error is due to calibration pushing all the peaks up or due to faulty data collection/ material. In the 2nd experiment, we had to perform 2 a spacial and a spectral calibration to end up with the corrected spectrum which then was used to determine the peak wavelength intensity of various stars and confirm that the temperature is higher or lower depending on star types. Unfortunately without further research we could not analyze the full chemical composition of the chosen stars, but we were able to confirm the presence of hydrogen, especially in lower temperature stars.
to extend this experiment, we would first add an O type star to have 1 star of each type within the experiment. Secondly, we would make further analysis of individual element spectra and star spectra to determine the chemical composition and element concentration in different star types. Finally, we want to reduce noise much more than. we have in this experiment. Measurement were not perfect but through better machinery or longer exposure times, i could allow use to have better and more accurate data.

# 7 References

Cowan, G. 2023, Statistical Data Analysis (Clarendon)10
Cannon, A. J., Pickering, E., Maury, A., and Fleming, W. (n.d.). Types of stars. Las Cumbres Observatory. https://lco.global/spacebook/stars/types-stars/

# 8 Appendix

## 8.1 Experiment 1 code

```
# Import dependencies
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import matplotlib
import scipy as sp
from scipy.signal import find_peaks
from scipy.optimize import curve_fit

folder_path = r'/Users/adrienbonansea/Downloads/SpectroF_Nov7/'
reference_path = r'/Users/adrienbonansea/Downloads/01_NIST.asc'

def stacked_ambience(folder_path, num_images=30):
    all_intensities = []

    for i in range(num_images):
        file_name = f'GroupF_Nov_7_Cap_USB2G560811__{i}__{60 + i:03d}.txt'
        file_path = f'{folder_path}{file_name}'

        data = np.loadtxt(file_path, skiprows=13)

        intensities = data[:, 1]  # Intensities (second column)

        all_intensities.append(intensities)

    all_intensities = np.array(all_intensities)

    averaged_data = np.mean(all_intensities, axis=0)

    wavelengths = data[:, 0]  # Wavelengths from the last loaded file

    averaged_spectrum = np.column_stack((wavelengths, averaged_data))

    return averaged_spectrum

def stacked_read_noise(folder_path, num_images=30):
    all_intensities = []

    for i in range(num_images):
        file_name = f'GroupF_Nov_7_Off_USB2G560811__{i}__{30 + i:03d}.txt'
        file_path = f'{folder_path}{file_name}'

        data = np.loadtxt(file_path, skiprows=13)

        intensities = data[:, 1]
        all_intensities.append(intensities)

    all_intensities = np.array(all_intensities)

    averaged_data = np.mean(all_intensities, axis=0)

    wavelengths = data[:, 0]
    averaged_spectrum = np.column_stack((wavelengths, averaged_data))

    return averaged_spectrum
```

```
def stacked_light(folder_path, num_images=30):
    all_intensities = []

    for i in range(num_images):
        file_name = f'GroupF_Nov_7_H2_USB2G560811__{i}__{00 + i:03d}.txt'
        file_path = f'{folder_path}{file_name}'

        data = np.loadtxt(file_path, skiprows=13)

        intensities = data[:, 1]

        all_intensities.append(intensities)

    all_intensities = np.array(all_intensities)

    averaged_data = np.mean(all_intensities, axis=0)

    wavelengths = data[:, 0]

    averaged_spectrum = np.column_stack((wavelengths, averaged_data))

    return averaged_spectrum

def subtract_noise_and_ambience(folder_path):
    st_read_noise = stacked_read_noise(folder_path)
    st_ambience = stacked_ambience(folder_path)
    st_light = stacked_light(folder_path)

    light_intensities = st_light[:, 1]
    read_noise_intensities = st_read_noise[:, 1]
    ambience_intensities = st_ambience[:, 1]

    corrected_light_intensity = light_intensities - read_noise_intensities - ambience_intensities

    corrected_floor_light = np.maximum(corrected_light_intensity, 0)
    wavelengths = st_light[:, 0]

    corrected_light = np.column_stack((wavelengths, corrected_floor_light))

    return corrected_light

def plot_corrected_light(corrected_light):
    wavelengths = corrected_light[:, 0]
    corrected_intensity = corrected_light[:, 1]
    normalized_intensities = corrected_intensity / np.max(corrected_intensity)

    plt.figure(figsize=(10, 6))
    plt.plot(wavelengths, normalized_intensities, label="Corrected Light Intensity")
    plt.xlabel("Wavelength [nm]")
    plt.ylabel("Normalized Intensity")
    plt.title("Corrected Spec of H")
    plt.grid(ls='-.')
    plt.savefig('ObservedH.jpg')
    plt.show()

def read_reference_data(reference_path):
    data = np.loadtxt(reference_path)
    return data

def plot_reference_data(reference_path):
```

```
    data = np.loadtxt(reference_path)

    wavelengths = data[:, 0]
    intensities = data[:, 1]

    plt.figure(figsize=(10, 6))
    plt.plot(wavelengths, intensities, label='Reference Spectrum')
    plt.xlabel('Wavelength [nm]')
    plt.ylabel('Normalized Intensity')
    plt.title('Reference Spec of H from NIST')
    plt.grid(ls='-.')
    plt.savefig('ReferenceH.jpg')
    plt.show()

def find_spectrum_peaks(wavelengths, intensities, height=None, prominence=None, distance=None):
    peaks, _ = find_peaks(intensities, height=height, prominence=prominence, distance=distance)
    return peaks

def plot_with_centroids(wavelengths, intensities, peaks, range_val=1):
    num_peaks = len(peaks)
    colours = cm.plasma(np.linspace(0, 1, num_peaks))

    plt.figure(figsize=(10, 6))
    plt.plot(wavelengths, intensities, alpha=0.7, label='Data')
    plt.plot(wavelengths[peaks], intensities[peaks], 'x', color='coral', label='Peaks', ms=5)

    for i in range(num_peaks):
        plt.axvline(wavelengths[peaks[i] - range_val], color=colours[i], alpha=0.5)
        plt.axvline(wavelengths[peaks[i] + range_val], color=colours[i], alpha=0.5)

    plt.xlabel("Wavelength [nm]")
    plt.ylabel("Intensity")
    plt.title("Corrected Spectrum")
    plt.legend()
    plt.grid(ls='-.')
    plt.show()

def polyfit_spectral_lines(observed_wavelengths, known_wavelengths, degree):
    coeffs = np.polyfit(observed_wavelengths, known_wavelengths, degree)
    polynomial = np.poly1d(coeffs)

    predicted_wavelengths = polynomial(observed_wavelengths)
    residuals = known_wavelengths - predicted_wavelengths


    chi_squared = np.sum((residuals ** 2))
    reduced_chi_squared = chi_squared / (len(observed_wavelengths) - (degree + 1))

    fig, ax = plt.subplots(2, 1, figsize=(12, 10), gridspec_kw={'height_ratios': [3, 1]})

    ax[0].scatter(
        observed_wavelengths, known_wavelengths, s=100, label='Observed Data'
    )
    ax[0].plot(
        np.sort(observed_wavelengths),
        polynomial(np.sort(observed_wavelengths)),
        color='black', alpha=0.7,
        label=(f'Fit: degree {degree}\n'
               f'Chi-Sq: {chi_squared:.2f}\n'
               f'Reduced Chi-Sq: {reduced_chi_squared:.2f}'
```

```
        )
    )
    ax[0].set_xlabel('Observed Wavelength [nm]', fontsize=12)
    ax[0].set_ylabel('Known Wavelength [nm]', fontsize=12)
    ax[0].set_title('Spectral Lines', fontsize=16)
    ax[0].legend(fontsize=10, loc='upper center', bbox_to_anchor=(0.15, 1))
    ax[0].grid(ls='-.', alpha=0.5)

    ax[1].scatter(
        observed_wavelengths, residuals,
        color='red', s=100
    )
    ax[1].axhline(0, color='black', alpha=0.7)
    ax[1].set_xlabel('Observed Wavelength [nm]', fontsize=12)
    ax[1].set_ylabel('Residuals [nm]', fontsize=12)
    ax[1].set_title('Residuals', fontsize=16)
    ax[1].grid(ls='-.', alpha=0.5)

    plt.subplots_adjust(hspace=0.5)  # Add space between the two plots
    plt.tight_layout(pad=4)  # Add padding around the figure
    plt.savefig(f'H (n ={degree}).jpg')
    plt.show()

    print(f"Coefficients: {coeffs}")
    print(f"Chi-Squared: {chi_squared:.2f}")
    print(f"Reduced Chi-Squared: {reduced_chi_squared:.2f}")
    return coeffs, chi_squared, reduced_chi_squared

corrected = subtract_noise_and_ambience(folder_path)
wavelengths = corrected[:, 0]
intensities = corrected[:, 1]
plot_corrected_light(corrected)

height_threshold = 10
prominence_threshold = 10
distance_between_peaks = 1
peaks = find_spectrum_peaks(wavelengths, intensities, height=height_threshold, prominence=prominen



reference = read_reference_data(reference_path)
wavelengths_ref = reference[:, 0]
intensities_ref = reference[:, 1]
plot_reference_data(reference_path)

height_threshold_ref = 0.0001
prominence_threshold_ref = 0.007
distance_between_peaks_ref = 1
peaks_ref = find_spectrum_peaks(wavelengths_ref, intensities_ref, height=height_threshold_ref, pro


observed_wavelengths = wavelengths[peaks]
known_wavelengths = wavelengths_ref[peaks_ref]  spectral line wavelengths (nm)

print(observed_wavelengths)
print(known_wavelengths)

index_observed = [0, 1, 3]
index_known = [2, 3, 4]
```

```
observed_wavelengths_reduct = observed_wavelengths[index_observed]
known_wavelengths_reduct = known_wavelengths[index_known]

degrees = np.arange(2)
for degree in degrees:
    polyfit_spectral_lines(observed_wavelengths_reduct, known_wavelengths_reduct, degree)

#do the same but replace files with neon files
```

## 8.2 Experiment 2 code

```
def process_dark_frames(file, exp_time):
averaged_darks = {}
for exp_time in exp_time:
folder_path = file / exp_time
fit_files = list(folder_path.glob('*.fit'))
{ 14 {
images = []
for fit_file in fit_files:
images.append(read_fits_file(fit_file))
averaged_dark = average_images(images)
averaged_darks[exp_time] = averaged_dark
return averaged_darks
def process_flat_frames(file):
flat_fit_files = list(file.glob('*.fit'))
images = []
for fit_file in flat_fit_files:
images.append(read_fits_file(fit_file))
averaged_flat = average_images(images)
plt.figure(figsize=(8, 6))
plt.imshow(averaged_flat, cmap='plasma')
plt.colorbar(label='Counts')
plt.title('Averaged Flat Frame')
plt.xlabel('Pixel X')
plt.ylabel('Pixel Y')
plt.tight_layout()
plt.show()
return averaged_flat
```